

Revision 2026-10-06 — Preparation and submission.en

1. Receiving new market data from IBKR

Market data reception — IB thread: the callback constructs `TradeReceived` and places it in the canonical queue.

```
TradeReceived:
    run_id="TEST_RUN"
    event_id="EVT_00001"
    symbol="MNQZ6"
    timestamp_utc=datetime(...)
    price=29898.50
    size=1.0
```

2. Canonical thread: `TradeReceivedHandler.handle(event)`

Responsibilities:

- Update the MarketStore
- Update the indicators
- Check that the context is ready for a decision
- Call `strategy.decide(context)`
- Return `IntentGenerated` if applicable

```
IntentGenerated(
    run_id="TEST_RUN",
    event_id="EVT_00002",
    causation_id="EVT_00001",
    instrument_id="MNQZ6",
    order_spec=BracketOrderSpec(
        entry=MarketOrderSpec(
            side=BUY,
            quantity=1.0,
            tif="DAY",
        ),
        stop_loss=StopOrderSpec(
            side=SELL,
            quantity=1.0,
            stop_price=29800,
            tif="DAY",
        ),
        take_profit=LimitOrderSpec(
            side=SELL,
            quantity=1.0,
```

```

        limit_price=30200,
        tif="DAY",
    ),
),
)

```

Element	V1 scope
Instrument	One instrument per intent
Entry	Market or Limit, BUY or SELL
Exits	An optional Stop Market order and an optional Limit order
Quantities	Each configured exit covers the full entry quantity
Activation	After the entry is fully filled
Coordination	A fully filled exit triggers cancellation of the other
Entry canceled without any fills	Discard the exit orders
Partial fills	Outside the scenarios validated in V1

Allowed variants:

- entry : MarketOrderSpec or LimitOrderSpec .
- stop_loss : StopOrderSpec or None .
- take_profit : LimitOrderSpec or None .

3. Canonical thread: IntentGeneratedHandler.handle(event)

Responsibilities:

- Split the intent into 2 or 3 OrderToPrepare objects
- Allocate an internal ID to each order
- Establish the internal parent-child relationship
- Return the commands to execute

```

prepare=PrepareBracket(
    run_id="TEST_RUN",
    command_id="CMD_00001",

    intention_id=event.event_id,
    instrument_id=event.instrument_id,

    entry=OrderToPrepare(

```

```

        order_id=self._order_id_generator.next_id(),
        parent_id=None,
        spec=event.order_spec.entry
    ),

    stop_loss=OrderToPrepare(
        order_id=self._order_id_generator.next_id(),
        parent_id=entry.order_id,
        spec=event.order_spec.stop_loss
    ),

    take_profit=OrderToPrepare(
        order_id=self._order_id_generator.next_id(),
        parent_id=entry.order_id,
        spec=event.order_spec.take_profit
    ),
)

return HandlerResult(
    events=(),
    commands=(prepare,),
)

```

4. Canonical thread: `PrepareBracket.handle(command)`

Responsibilities:

- Schedule order preparation on the IB event loop via `call_soon_threadsafe(...)`
- Return without waiting.

```

self._ib_client.ib_loop.call_soon_threadsafe(
    self._ib_adapter.prepare_on_ib_thread,
    command,
)

```

5. IB thread: `ib_adapter.prepare_on_ib_thread(command)`

- Correlate the event with the intent and preparation IDs
- For each order in the bracket, create `PrepareOrder` with:
 - internal IDs (order and parent)
 - broker IDs (order and parent)
 - the value of the flag `transmit`,
- Place `BracketPrepared` in the internal canonical queue for processing.

```

prepared = BracketPrepared(
    run_id=command.run_id,

```

```

event_id="EVT_00003"

intention_id=command.intention_id,
preparation_id=command.command_id,
instrument_id=command.instrument_id,

entry=PreparedOrder(
    order_id=command.entry.order_id,
    parent_id=command.entry.parent_id,
    broker_order_id=278,
    broker_parent_id=0,
    spec=command.entry.spec,
    transmit=False,
),

stop_loss=PreparedOrder(
    order_id=command.stop_loss.order_id,
    parent_id=command.stop_loss.parent_id,
    broker_order_id=279,
    broker_parent_id=278,
    spec=command.stop_loss.spec,
    transmit=False if command.take_profit else True,
),

take_profit=PreparedOrder(
    order_id=command.take_profit.order_id,
    parent_id=command.take_profit.parent_id,
    broker_order_id=280,
    broker_parent_id=278,
    spec=command.take_profit.spec,
    transmit=True,
),
)

internal_event_queue.enqueue(prepared)

```

6. Canonical thread: `BracketPreparedHandler.handle(event)`

Responsibilities:

- Create the canonical `OrderCreated` event so that the projector can build the order view.
- Create an `IBSubmitOrder` command for each order.

Entry — internal order 1, broker order 278

```

entry_created = OrderCreated(
    run_id=event.run_id,
    event_id="EVT_00004",
    intention_id=event.intention_id,

```

```

        instrument_id=event.instrument_id,
        order_id=event.entry.order_id,                # 1
        parent_id=event.entry.parent_id,             # None
        broker_order_id=event.entry.broker_order_id, # 278
        broker_parent_id=event.entry.broker_parent_id, # 0
        spec=event.entry.spec,                        # ENTRY
    )

    submit_entry = IBSubmitOrder(
        run_id=event.run_id,
        command_id="CMD_00002",
        instrument_id=event.instrument_id,
        order_spec=event.entry.spec,
        broker_order_id=event.entry.broker_order_id,
        broker_parent_id=event.entry.broker_parent_id,
        transmit=event.entry.transmit,                # False
    )

```

StopLoss — internal order 2 , broker order 279

```

    stop_loss_created = OrderCreated(
        run_id=event.run_id,
        event_id="EVT_00005",
        intention_id=event.intention_id,
        instrument_id=event.instrument_id,
        order_id=event.stop_loss.order_id,            # 2
        parent_id=event.stop_loss.parent_id,          # 1
        broker_order_id=event.stop_loss.broker_order_id, # 279
        broker_parent_id=event.stop_loss.broker_parent_id, # 278
        spec=event.stop_loss.spec,                    # STOP_LOSS
    )

    submit_stop_loss = IBSubmitOrder(
        run_id=event.run_id,
        command_id="CMD_00003",
        instrument_id=event.instrument_id,
        order_spec=event.stop_loss.spec,
        broker_order_id=event.stop_loss.broker_order_id,
        broker_parent_id=event.stop_loss.broker_parent_id,
        transmit=event.stop_loss.transmit,            # False
    )

```

TakeProfit — internal order 3 , broker order 280

```

    take_profit_created = OrderCreated(
        run_id=event.run_id,
        event_id="EVT_00006",
        intention_id=event.intention_id,

```

```

        instrument_id=event.instrument_id,
        order_id=event.take_profit.order_id,           # 3
        parent_id=event.take_profit.parent_id,        # 1
        broker_order_id=event.take_profit.broker_order_id, # 280
        broker_parent_id=event.take_profit.broker_parent_id, # 278
        spec=event.take_profit.spec,                  # TAKE_PROFIT
    )

    submit_take_profit = IBSubmitOrder(
        run_id=event.run_id,
        command_id="CMD_00004",
        instrument_id=event.instrument_id,
        order_spec=event.take_profit.spec,
        broker_order_id=event.take_profit.broker_order_id,
        broker_parent_id=event.take_profit.broker_parent_id,
        transmit=event.take_profit.transmit,          # True
    )

```

```

return HandlerResult(
    events=(entry_created, stop_loss_created, take_profit_created),
    commands=(submit_entry, submit_stop_loss, submit_take_profit),
)

```

7. Processing the result in the canonical loop

Responsibility:

- Append canonical events to the journal
- Update the correlation table
- Enqueue the `IBSubmitOrder` commands in order in the internal queue for processing.

1. Append the three `OrderCreated` events to the journal.
2. Update the correlation table:
 - broker 278 ↔ internal 1
 - broker 279 ↔ internal 2
 - broker 280 ↔ internal 3
3. Dispatch the `IBSubmitOrder` commands: entry → stop-loss → take-profit

8. Canonical thread: `IBSubmitOrderHandler.handle(command)`

Responsibility: Schedule order submission on the IB thread

```

def handle(self, command):
    self._ib_client.ib_loop.call_soon_threadsafe(
        self._ib_adapter.submit_on_ib_thread,

```

```
        command,  
    )
```

9. IB thread: `ib_adapter.submit_on_ib_thread(command)`

Responsibilities:

- Resolve `command.instrument_id` to an IBKR contract,
- Convert the command into an IBKR order.
- Place the orders using `ib_insync ib.placeOrder(instrument, order)`

```
def submit_on_ib_thread(self, command):  
    contract = self.resolve_contract(command.instrument_id)  
  
    ib_order = self.convert_order(  
        spec=command.order_spec,  
        order_id=command.broker_order_id,  
        parent_id=command.broker_parent_id,  
        transmit=command.transmit,  
    )  
  
    self._ib_client.ib.placeOrder(contract, ib_order)
```